

# TCP Server Solution with Explanation

## Java TCP Echo Server Code:

```
import java.net.*;
import java.io.*;

public class TCPServer {

    public static void main(String[] args) {

        try {

            // Create server socket on port 7733
            ServerSocket server =
                new ServerSocket(7733);

            System.out.println("Server started...");

            // Keep server running forever
            while (true) {

                // Accept client connection
                Socket nextClient = server.accept();

                System.out.println("Client connected: "
                    + nextClient.getInetAddress());

                // Input stream to receive message
                BufferedReader reader =
                    new BufferedReader(
                        new InputStreamReader(
                            nextClient.getInputStream()));

                // Output stream to send message
                PrintStream pout =
                    new PrintStream(
                        nextClient.getOutputStream());

                // Read message from client
                String msg = reader.readLine();

                System.out.println("Received: " + msg);

                // Send same message back to client
                pout.println(msg);

                // Close client socket
                nextClient.close();
            }

        } catch (IOException ioe) {

            System.out.println("Server Error: " + ioe);

        }

    }

}
```

## Step-by-Step Explanation:

1. `ServerSocket server = new ServerSocket(7733);`  
Opens port 7733 and waits for clients.
2. `while(true)`  
Keeps the server running forever.
3. `Socket nextClient = server.accept();`  
Waits for a client connection.  
`accept()` is a blocking method.
4. `BufferedReader reader ...`  
Used to read text sent by the client.
5. `PrintStream pout ...`  
Used to send text to the client.
6. `String msg = reader.readLine();`  
Reads the message sent by the client.
7. `pout.println(msg);`  
Sends the same message back to the client.  
This is why it is called an Echo Server.
8. `nextClient.close();`  
Closes the client connection.
9. `catch(IOException ioe)`  
Handles any network/input-output errors.